Atividade: Reconhecimento de Gestos com Deep Learning

Disciplina de Machine Learning e Deep Learning

Contexto

Em aula, usamos uma CNN com transfer learning (MobileNetV2) para reconhecer gestos de pedra, papel e tesoura pela webcam e usá-los para jogar contra o computador. Esta atividade tem cinco partes: coletar dados, treinar o modelo, jogar com ele, desenvolver o trabalho com sinais de Libras e responder a quatro perguntas descritivas sobre o que foi feito.

Parte 1: Coleta de dados

Requisitos: Python 3 e a biblioteca OpenCV (`pip install opencv-python`).

Como usar:

1. Salve o código abaixo em um arquivo chamado `collect.py`.
2. Execute `python collect.py` no terminal.
3. Clique uma vez na **janela do vídeo** (não no terminal) para ela receber as teclas.
4. Aperte **uma vez** (sem segurar) a tecla do gesto: `1` ou `p` = pedra, `2` ou `a` = papel, `3` ou `t` = tesoura, `4` ou `n` = nada (sem mão).
5. Uma contagem de 3 segundos aparece para você posicionar a mão dentro do quadrado. Depois a gravação começa sozinha (quadrado vermelho).
6. Aperte a mesma tecla para pausar, ou outra para mudar de gesto. Aperte `q` para sair.

As fotos são salvas na pasta `data/`, criada no mesmo local de onde você executou o comando, com uma subpasta para cada gesto (`pedra`, `papel`, `tesoura`, `nada`).

Dica: varie o fundo, a luz, a distância, o ângulo e a mão (direita e esquerda). A variedade dos dados influencia muito o resultado do modelo.

Código: collect.py

```
"""
Passo 1 - Coleta de dados pela webcam.

Uso:  python collect.py

IMPORTANTE: clique uma vez na JANELA DO VIDEO para ela receber as teclas
(se o terminal estiver em foco, nada acontece).

Teclas (aperte UMA vez, nao segure):
  1 ou p = pedra     2 ou a = papel     3 ou t = tesoura     4 ou n = nada
  Ao apertar, ha uma contagem de 3s para voce posicionar a mao, e a
  gravacao comeca sozinha. Aperte a mesma tecla de novo para pausar.
  q = sair
"""
import os
import time
import cv2

KEYS = {
    ord("1"): "pedra", ord("p"): "pedra",
    ord("2"): "papel", ord("a"): "papel",
```

```
    ord("3"): "tesoura", ord("t"): "tesoura",
    ord("4"): "nada", ord("n"): "nada",
}
CLASSES = ["pedra", "papel", "tesoura", "nada"]
DATA_DIR = "data"
IMG_SIZE = 224
SAVE_EVERY_N_FRAMES = 3
MAX_PER_CLASS = 150
COUNTDOWN = 3.0
DEBOUNCE = 0.5   # ignora repeticao de tecla (segurar) por 0.5s

for c in CLASSES:
    os.makedirs(os.path.join(DATA_DIR, c), exist_ok=True)

def count(cls):
    return len([f for f in os.listdir(os.path.join(DATA_DIR, cls)) if f.endswith(".jpg")])

cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise SystemExit("Nao consegui abrir a webcam. Feche outros apps que usam a camera "
                     "(Zoom, Meet...) ou tente cv2.VideoCapture(1).")

recording = None       # classe sendo gravada
countdown_start = None
last_key_time = 0
frame_count = 0
WIN = "Coleta"
print("Janela aberta. Clique nela e aperte 1/2/3/4 (ou p/a/t/n). 'q' sai.")

while True:
    ok, frame = cap.read()
    if not ok:
        print("Falha ao ler a webcam.")
        break
    frame = cv2.flip(frame, 1)
    h, w = frame.shape[:2]
    side = min(h, w) // 2 + 60
    x1, y1 = w // 2 - side // 2, h // 2 - side // 2
    roi = frame[y1:y1 + side, x1:x1 + side]

    status, color = "PARADO - aperte 1/2/3/4 na janela", (0, 255, 0)
    if recording and countdown_start is not None:
        left = COUNTDOWN - (time.time() - countdown_start)
        if left > 0:
            status, color = f"{recording}: prepare a mao... {int(left) + 1}", (0, 255, 255)
        else:
            countdown_start = None
    if recording and countdown_start is None:
        status, color = f"GRAVANDO: {recording}", (0, 0, 255)
        frame_count += 1
        if frame_count % SAVE_EVERY_N_FRAMES == 0:
            if count(recording) < MAX_PER_CLASS:
                img = cv2.resize(roi, (IMG_SIZE, IMG_SIZE))
                cv2.imwrite(os.path.join(DATA_DIR, recording, f"{int(time.time() * 1000)}.jpg"), img)
            else:
                print(f"{recording}: completo ({MAX_PER_CLASS} fotos).")
                recording = None

    cv2.rectangle(frame, (x1, y1), (x1 + side, y1 + side), color, 3)
    cv2.putText(frame, status, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)
    for i, c in enumerate(CLASSES):
        cv2.putText(frame, f"[{i + 1}/{c[0]}] {c}: {count(c)}/{MAX_PER_CLASS}", (10, 65 + i * 28),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)
    cv2.imshow(WIN, frame)

    key = cv2.waitKey(10) & 0xFF
    if key == 255:
        continue
    if key == ord("q"):
        break
    if key in KEYS and time.time() - last_key_time > DEBOUNCE:
        last_key_time = time.time()
        cls = KEYS[key]
        if recording == cls:
            recording = None
            print(f"Pausado: {cls}")
        else:
            recording, countdown_start = cls, time.time()
```

```
            print(f"Preparando: {cls}")
    elif key not in KEYS:
        print(f"(tecla nao reconhecida: codigo {key})")

cap.release()
cv2.destroyAllWindows()
print({c: count(c) for c in CLASSES})
```

Parte 2: Treino do modelo

Requisitos: instale também o TensorFlow: `pip install tensorflow numpy matplotlib opencv-python`.

Como usar:

1. Salve o código abaixo como `train.py`, na mesma pasta onde está a pasta `data`.
2. Execute `python train.py`.
3. Ao final, o script salva o modelo (`gesture_model.keras`), a lista de classes (`class_names.json`) e o gráfico `curvas.png`, e imprime a matriz de confusão no terminal.

O treino usa transfer learning: a MobileNetV2, já treinada na ImageNet, fica congelada, e só a camada final nova é treinada para as suas classes. A validação usa os últimos 20% das fotos de cada classe, na ordem em que foram coletadas.

Código: train.py

```
"""
Passo 2 - Treino com Transfer Learning (MobileNetV2).

Uso:  python train.py

Ideia: a base (MobileNetV2, pre-treinada na ImageNet) ja sabe "enxergar"
bordas, texturas e formas. Congelamos ela e treinamos so a "cabeca" nova
que decide entre as classes encontradas na pasta data.
"""
import os
import json
import random
import numpy as np
import tensorflow as tf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

DATA_DIR = "data"
IMG_SIZE = 224
BATCH = 16
EPOCHS = 8
VAL_FRACTION = 0.2
SEED = 42
random.seed(SEED); np.random.seed(SEED); tf.random.set_seed(SEED)

class_names = sorted(d for d in os.listdir(DATA_DIR)
                     if os.path.isdir(os.path.join(DATA_DIR, d)))
print("Classes:", class_names)

# Split por BLOCOS no tempo (ultimos 20% de cada classe = validacao).
# Se sorteassemos aleatoriamente, frames quase identicos cairiam em treino e
# validacao e a acuracia ficaria enganosamente alta (vazamento de dados).
train_paths, train_y, val_paths, val_y = [], [], [], []
for idx, cls in enumerate(class_names):
    files = sorted(os.listdir(os.path.join(DATA_DIR, cls)))
    files = [os.path.join(DATA_DIR, cls, f) for f in files if f.lower().endswith(".jpg")]
    cut = int(len(files) * (1 - VAL_FRACTION))
    train_paths += files[:cut]; train_y += [idx] * cut
    val_paths += files[cut:];   val_y += [idx] * (len(files) - cut)
    print(f"  {cls}: {cut} treino / {len(files) - cut} validacao")

def load(path, label):
```

```
    img = tf.io.read_file(path)
    img = tf.image.decode_jpeg(img, channels=3)
    img = tf.image.resize(img, (IMG_SIZE, IMG_SIZE))
    return img, label

def make_ds(paths, labels, training):
    ds = tf.data.Dataset.from_tensor_slices((paths, labels))
    if training:
        ds = ds.shuffle(len(paths), seed=SEED)
    ds = ds.map(load, num_parallel_calls=tf.data.AUTOTUNE).batch(BATCH)
    return ds.prefetch(tf.data.AUTOTUNE)

train_ds = make_ds(train_paths, train_y, True)
val_ds = make_ds(val_paths, val_y, False)

# Data augmentation: so atua durante o treino
augment = tf.keras.Sequential([
    tf.keras.layers.RandomRotation(0.08),
    tf.keras.layers.RandomZoom(0.15),
    tf.keras.layers.RandomBrightness(0.25),
    tf.keras.layers.RandomContrast(0.25),
])

base = tf.keras.applications.MobileNetV2(
    input_shape=(IMG_SIZE, IMG_SIZE, 3), include_top=False, weights="imagenet")
base.trainable = False   # <- o "congelamento" do transfer learning

inputs = tf.keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
x = augment(inputs)
x = tf.keras.applications.mobilenet_v2.preprocess_input(x)
x = base(x, training=False)
x = tf.keras.layers.GlobalAveragePooling2D()(x)
x = tf.keras.layers.Dropout(0.3)(x)
outputs = tf.keras.layers.Dense(len(class_names), activation="softmax")(x)
model = tf.keras.Model(inputs, outputs)

model.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.summary()

hist = model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS)

model.save("gesture_model.keras")
with open("class_names.json", "w") as f:
    json.dump(class_names, f)

# Curvas de treino vs validacao (otimo para discutir overfitting em aula)
plt.figure(figsize=(10, 4))
for i, (m, t) in enumerate([("accuracy", "Acuracia"), ("loss", "Loss")]):
    plt.subplot(1, 2, i + 1)
    plt.plot(hist.history[m], label="treino")
    plt.plot(hist.history["val_" + m], label="validacao")
    plt.title(t); plt.xlabel("epoca"); plt.legend()
plt.tight_layout(); plt.savefig("curvas.png")

# Matriz de confusao simples
y_true = np.array(val_y)
y_pred = np.argmax(model.predict(val_ds, verbose=0), axis=1)
cm = np.zeros((len(class_names), len(class_names)), dtype=int)
for t, p in zip(y_true, y_pred):
    cm[t, p] += 1
print("\nMatriz de confusao (linhas = real, colunas = previsto):")
print("            " + "  ".join(f"{c[:7]:>7}" for c in class_names))
for i, c in enumerate(class_names):
    print(f"{c[:10]:>10}  " + "  ".join(f"{v:>7}" for v in cm[i]))
print("\nPronto! Modelo salvo em gesture_model.keras. Agora rode: python play.py")
```

Parte 3: Jogo (pedra, papel e tesoura)

Como usar:

1. Salve o código abaixo como `play.py`, na mesma pasta de `gesture_model.keras` e `class_names.json`.
2. Execute `python play.py` e clique na janela do vídeo.

3 Aperte ESPAÇO para começar uma rodada: após a contagem 3-2-1, o modelo lê a sua mão e o computador joga.

4 Aperte q para sair.

Na tela aparecem o gesto que o modelo está vendo, a confiança da previsão e uma barra de probabilidade para cada classe, o que ajuda a perceber quando o modelo está em dúvida.

Código: play.py

```
"""
Passo 3 - Jogo Pedra, Papel e Tesoura contra o computador.

Uso:  python play.py
  ESPACO = nova rodada (contagem 3-2-1, depois o modelo "le" sua mao)
  q      = sair
"""
import json
import random
import time
from collections import Counter, deque

import cv2
import numpy as np
import tensorflow as tf

IMG_SIZE = 224
model = tf.keras.models.load_model("gesture_model.keras")
class_names = json.load(open("class_names.json"))

BEATS = {"pedra": "tesoura", "papel": "pedra", "tesoura": "papel"}  # chave vence valor
MOVES = ["pedra", "papel", "tesoura"]

def predict(roi):
    img = cv2.cvtColor(cv2.resize(roi, (IMG_SIZE, IMG_SIZE)), cv2.COLOR_BGR2RGB)
    probs = model.predict(img[None].astype("float32"), verbose=0)[0]
    return class_names[int(np.argmax(probs))], float(np.max(probs)), probs

cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise SystemExit("Nao consegui abrir a webcam.")

recent = deque(maxlen=7)          # suaviza: voto da maioria nos ultimos frames
score = {"voce": 0, "pc": 0, "empates": 0}
state, t0, message, pc_move = "idle", 0, "Aperte ESPACO para jogar", None

while True:
    ok, frame = cap.read()
    if not ok:
        break
    frame = cv2.flip(frame, 1)
    h, w = frame.shape[:2]
    side = min(h, w) // 2 + 60
    x1, y1 = w // 2 - side // 2, h // 2 - side // 2
    roi = frame[y1:y1 + side, x1:x1 + side]

    label, conf, probs = predict(roi)
    recent.append(label)
    stable = Counter(recent).most_common(1)[0][0]

    if state == "count":
        left = 3 - int(time.time() - t0)
        if left <= 0:
            you = stable
            if you not in MOVES:
                message = "Nao vi sua mao! Tente de novo"
            else:
                pc_move = random.choice(MOVES)
                if you == pc_move:
                    score["empates"] += 1; message = f"Empate! PC: {pc_move}"
                elif BEATS[you] == pc_move:
                    score["voce"] += 1; message = f"Voce ganhou! ({you} x {pc_move})"
                else:
                    score["pc"] += 1; message = f"PC ganhou! ({you} x {pc_move})"
            state, t0 = "result", time.time()
```

```
            else:
                message = f"{left}..."
        elif state == "result" and time.time() - t0 > 3:
            state = "idle"

        cv2.rectangle(frame, (x1, y1), (x1 + side, y1 + side), (0, 255, 0), 3)
        cv2.putText(frame, f"Modelo ve: {label} ({conf:.0%})", (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 0), 2)
        # barras de probabilidade (otimo para mostrar a "incerteza" do modelo)
        for i, c in enumerate(class_names):
            bw = int(probs[i] * 150)
            cv2.rectangle(frame, (10, 50 + i * 22), (10 + bw, 64 + i * 22), (0, 200, 255), -1)
            cv2.putText(frame, c, (170, 63 + i * 22), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)
        cv2.putText(frame, message, (10, h - 50), cv2.FONT_HERSHEY_SIMPLEX, 1.0, (0, 255, 0), 3)
        cv2.putText(frame, f"Voce {score['voce']} x {score['pc']} PC  (empates: {score['empates']})",
                    (10, h - 15), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)
        cv2.imshow("Pedra, Papel e Tesoura com CNN", frame)

        key = cv2.waitKey(1) & 0xFF
        if key == ord("q"):
            break
        if key == 32 and state == "idle":
            state, t0 = "count", time.time()

cap.release()
cv2.destroyAllWindows()
```

Parte 4: Trabalho com sinais de Libras

Agora é a sua vez de aplicar tudo isso em um problema novo. O trabalho pode ser feito em **trios**: o grupo deve treinar um modelo que reconheça **10 sinais de Libras**, com dados coletados por vocês.

O que fazer:

1. Escolham 10 sinais de Libras. Prefiram sinais **estáticos** (sem movimento), porque o modelo analisa uma foto por vez, e confirmem a forma correta de cada sinal em uma fonte confiável de Libras.
2. Adaptem o `collect.py`: troquem a lista de classes pelos nomes dos seus sinais e ajustem o mapa de teclas (com 10 sinais, usem as teclas de 1 a 9 e 0, ou letras). Mantenham a classe `nada`.
3. Coletem cada sinal em **duas iluminações: fundo escuro e fundo claro**.
4. Treinem o modelo com o `train.py`, que lê as classes direto das pastas dentro de `data`.
5. Adaptem o `play.py` para mostrar o sinal que o modelo reconhece, no lugar do jogo de pedra, papel e tesoura.

Cuidados com os dados:

- Mantenham um número parecido de fotos em todos os sinais. Como são duas iluminações, aumentem o valor de `MAX_PER_CLASS` no `collect.py` (por exemplo, 300 por sinal).
- Intercalem as duas iluminações durante a coleta de cada sinal, em vez de coletar toda uma e depois a outra. O `train.py` usa os últimos 20% das fotos de cada classe como validação, então coletar uma iluminação por último deixaria a validação só com ela.
- Variem também o ângulo, a distância e a mão usada (direita e esquerda).

Regra de entrega: o envio deve ser feito em **PDF**, contendo as imagens da pasta `data` separadas por classe: os **10 sinais de Libras**, cada um nas **2 iluminações** (fundo escuro e fundo claro), além da classe `nada`. Só serão aceitos trabalhos entregues com esse PDF de imagens. A dica de código abaixo gera o arquivo automaticamente.

Dica de código: gerar o PDF das imagens

Para montar o PDF da entrega sem trabalho manual, acrescente o trecho abaixo ao **final** do `train.py`. Ele percorre a pasta `data` e cria o arquivo `dataset_imagens.pdf`, com uma seção para cada classe (cada página mostra o nome da classe, o total de imagens e as miniaturas). Rode o treino normalmente e envie o PDF gerado. Se o arquivo ficar grande demais para enviar, diminua o valor de `qualidade` ou aumente `cols`.

```
# ---- DICA: gera um PDF com as imagens do dataset, separadas por classe ----
# Cole este trecho no FINAL do train.py (ele usa o DATA_DIR e o os ja definidos la).
from PIL import Image, ImageDraw, ImageFont


def gerar_pdf_dataset(data_dir="data", saida="dataset_imagens.pdf", cols=10, qualidade=70):
    pag_w, pag_h, margem, topo = 1240, 1754, 40, 110   # pagina A4 a 150 dpi
    cell = (pag_w - 2 * margem) // cols                # tamanho de cada miniatura
    rows = (pag_h - topo - margem) // cell
    por_pagina = cols * rows
    try:
        fonte = ImageFont.load_default(size=30)
    except TypeError:                                  # Pillow mais antigo
        fonte = ImageFont.load_default()

    paginas = []
    classes = sorted(d for d in os.listdir(data_dir)
                     if os.path.isdir(os.path.join(data_dir, d)))
    for classe in classes:                             # uma secao por classe
        pasta = os.path.join(data_dir, classe)
        fotos = sorted(f for f in os.listdir(pasta) if f.lower().endswith(".jpg"))
        n_pag = max(1, -(-len(fotos) // por_pagina))   # divisao arredondada para cima
        for p in range(n_pag):
            pagina = Image.new("RGB", (pag_w, pag_h), "white")
            draw = ImageDraw.Draw(pagina)
            draw.text((margem, 40),
                      f"Classe: {classe}  |  {len(fotos)} imagens  |  pagina {p + 1}/{n_pag}",
                      fill="black", font=fonte)
            for i, nome in enumerate(fotos[p * por_pagina:(p + 1) * por_pagina]):
                img = Image.open(os.path.join(pasta, nome)).convert("RGB")
                img.thumbnail((cell - 6, cell - 6))
                x = margem + (i % cols) * cell + 3
                y = topo + (i // cols) * cell + 3
                pagina.paste(img, (x, y))
            paginas.append(pagina)

    paginas[0].save(saida, save_all=True, append_images=paginas[1:],
                    quality=qualidade, resolution=150)
    print(f"PDF salvo: {saida} ({len(paginas)} paginas)")


gerar_pdf_dataset(DATA_DIR)
```

Parte 5: Perguntas

Responda com suas próprias palavras, de forma descritiva, usando exemplos do que você observou na prática.

1. Esse modelo é um deep learning ou um machine learning?

Justifique sua resposta com base na arquitetura usada no projeto.

2. Que características da imagem o modelo usa para distinguir os gestos?

Descreva quais características da mão você acredita que o modelo usa e proponha um experimento para testar a sua hipótese. O que você observou?

3. Que decisões na coleta de dados ajudam ou prejudicam o modelo?

Descreva duas decisões que você tomou ao coletar os dados e como cada uma pode influenciar o resultado do modelo.

4. Teste: como o modelo se comporta se você treinar com dados que têm pessoas no fundo?

Faça o experimento e descreva o que aconteceu com as previsões. Explique por que o modelo teve esse comportamento e proponha duas estratégias para melhorá-lo.